

Q1. QA Data Creation:

We ran a script to create QA data from the parsed documents we got from our crawler. Here, we read through each file, while skipping through files that have less than 50 characters. From here, we send the document to an OpenAI API with a strict system prompt asking for one question answer pair from the document, where the answer is short (between 1 and 5 words). Once we get all the pairs, we split into a questions file, an answers file, and a references file that has the document referenced by each question. For this QA dataset, we have 114 question-answer pairs, and have a 68% IAA (human annotators annotated on close to half of the data). Here are a few examples from the dataset:

Where did Randy H. Katz receive his B.A? **Cornell University**

What year was Stephen Tu awarded the Google Fellowship? **2018**

Who won the 2020 IEEE Marie Skłodowska-Curie Award? **Michael Lieberman**

When was Stuart Russell named an Honorary Fellow of Wadham College? **October 29, 2018**

Q2. Retrieval Corpus

We start by iterating through the parsed documents we got from the crawler. Then, we use a sliding window that is based on word counts (like a 200-word chunk size), and an overlap of chunks (we started with an overlap of 40 words). Then, we store the text chunks in a pandas dataframe and export that to a file called data_storage.json. Their high-dimensional vector representations were stored with faiss in eecs_ind.faiss. We manually inspected our retrieval corpus, filtering out empty documents and documents under 50 characters. We also moved to a hybrid search of dense and sparse retrieval, which we talk about in Q3. The reference corpus seems to perform better, and that may be because it's preserving the global context of the page. If the LLM sees the entire context or maybe it sees chunks grouped by things like URL, or like headers in HTML, then it keeps related information together, instead of how we chunked it.

Q3. RAG System

For the RAG Model, we started with an input of a list of questions from the questions file. We then used a hybrid retrieval approach, with both dense and sparse retrieval. For the dense retrieval part, we used the BAAI/bge-small-en-v1.5 model, converting the query into a vector and performing an IndexFlatIP Search, after which we normalized. For sparse retrieval, we used BM25Okapi, which is a keyword-based algorithm that finds exact word matches. We then used this in the get_context function, getting the top 8 results from the dense and the top 4 from the sparse, making sure to remove duplicate chunks. Then, we used a very specific system prompt to make the LLM act as a strict extraction model and used the llama-3.1-8b-instruct model as the LLM. We had the model return "Not available" if it couldn't find the answer in the provided corpus. We also chose to go with a hybrid approach after trying both dense and sparse retrieval approaches and not getting fully great results from either.

Q4. Ablations

Chunking size and overlap: We did grid search over chunk size and overlap (when chunking each website's text into sections before embedding them via the pre-selected embedding model). We did a grid search over chunk_size of [100, 300] words and overlap sizes of [20, 60] words (our base model was chunk_size = 200, overlap = 40). The smaller chunk sizes led to worse performance; we suspect this might be because if some question needs a specific chunk text[i: j] to answer, a smaller chunk size

increases the probability that $\text{text}[i:j]$ gets split into multiple segments during chunking, none of which contain all the information required to answer the question. Similarly, overlap size did not affect performance much. Increasing overlap will almost monotonically improve performance, though it can significantly hinder speed.

Results \ Ablations (chunk_size - overlap)	100-20	100-60	200-40	300-20	300-60
Dev F1	46.0%	46.1%	50.6%	58.9%	58.3%
Test F1	51.1%	61.3%	62.6%	60.3%	57.5%

Sentence-aware chunking: As described above, our first attempt was to chunk the text files for each webpage into segments of fixed length. However, sentence-aware chunking does this without breaking up sentences, which would theoretically decrease the chance of a specific chunk $\text{text}[i:j]$ getting split into separate segments. However, it seems this didn't really have an impact. Grid search can similarly be done for this chunking, with chunk sizes being [4, 6, 8] sentences, and overlap being [1, 2, 3] sentences.

Q5. Error analysis

Question: "How many EE courses is Kristofer Pister teaching in Spring 2027?" **Correct answer:** "7"; **predicted** "0"

Before changing our prompt to the base LLM to maximize for F1 score, we would have the model return "Not available" if the information required to answer the question was not part of the retrieved context. This was one of the questions that always had the prediction "Not available." Thus, this is a retrieval error (the EECS Spring 2027 draft schedule was likely not seen during crawling).

Question: "Who was the earliest-born professor at UC Berkeley?" **Correct answer:** "Lester Edwin Reukema"; **predicted** "1910"

This is an augmentation error, specifically, the model is not responding to the question with an appropriate format of answer (the question is asking for a person, whereas our model answered a year).

We dealt with metric limitations by creating a very specific LLM prompt that asked for no punctuation, provided exactly one answer when there were multiple valid choices, and limited the output to 10 words MAX. As a result, metric limitations were a very minimal percentage of errors.

Q6. Takeaways & Future Ideas

Our key takeaways were that retrieval is not too nitpicky; increasing chunking size to end up with fewer embeddings (to optimize for speed) is often a worthwhile tradeoff (since retrieval correctness isn't significantly affected). We can intuitively come to this conclusion when considering "bag of words." We did some research into reranking the top_k candidates (regardless of sparse, dense, or hybrid retrieval implementation) after determining who they are. In particular, our model currently simply concatenates the contents of the top_k candidates into a single "context" string and feeds that into the QA LLM.

GenAI contribution statement: We used Generative AI for questions about opening and writing into files, questions about if our prompt was specific enough, and questions about different embedding models and base models we were looking into.